مراجعة المختبر 1

C للبرمجة المتوازية: الترجمة، سطر الأوامر، الماكرو، والسكربتات

> **هدف هذا الملف**
> هذا الملف للمراجعة والفهم فقط، ماشي جوابات للتسليم.
> نفهمو علاش نديرو كل خطوة، شنو يصير في كل أمر، وشنو الأخطاء اللي تطيح فيها غالبا.

الفهرس

- الجزء 1: الصورة الكبيرة (علاش هذا المختبر)
- الجزء 2: التمرين 0، بيئة العمل
- الجزء 3: التمرين 1، من الكود للبرنامج التنفيذي
- الجزء 4: السكربتات والصلاحيات
- الجزء 5: التمرين 2، argc و argv
- الجزء 6: التمرين 3، atoi و strtol
- الجزء 7: التمرين 4، الماكرو
- الجزء 8: الأخطاء الشائعة وحلولها
- الجزء 9: قاموس المصطلحات
- الجزء 10: اختبر روحك
- الجزء 11: أمثلة كود للفهم

الجزء 1: الصورة الكبيرة

كل برامج المقرر القادم (MPI وPThreads وOpenMP وCUDA وOpenCL) هي برامج C. كلها تتترجم بعدة خطوات، تتشغل من سطر الأوامر، وتستعمل الـ preprocessor (#include و#define).

لهذا المختبر يراجع الأدوات الأساسية، باش في المختبرات الجاية تركز على التوازي ماشي على لغة C.

> **الفكرة المهمة**
> المختبر 1 يعلمك 3 حاجات: كيفاش يتترجم الكود، كيفاش يقرا البرنامج المعطيات من سطر الأوامر، وكيفاش تكتب سكربت يعاود الأوامر بأمر واحد.

الجزء 2: التمرين 0، بيئة العمل

الأدوات وعلاش نحتاجوهم

الأداة	شنو تدير	تستعملها في
`gcc`	مترجم لغة C	كل المختبرات
`mpicc / mpirun`	MPI: تشغيل برامج على عدة معالجات	`Lab 3`
`gcc -fopenmp`	OpenMP: التوازي بالـ threads	`Lab 4`
`clinfo`	يورينا واش خدام OpenCL (هنا PoCL)	`Lab 6`
`make`	يشغل Makefile ويبني المشروع	`Lab 2`

ملاحظة على الإصدارات

أرقام الإصدارات عندك قد تختلف عن PDF (gcc 15.2 و OpenMPI 6.0 مثلا). هذا عادي. المهم أن الأوامر تخدم.

خطأ mpirun --version

إذا ظهر لك "Sorry! You were supposed to get help about: version"، فهو يأثر على رسائل المساعدة فقط، وبرامج MPI تخدم عادي.

الجزء 3: التمرين 1، من الكود للبرنامج التنفيذي

3.1 الفكرة العامة

الكود اللي تكتبو في hello.c ما يتشغلش مباشرة. لازم يمر بأربع مراحل باش يولي ملف تنفيذي. كل مرحلة تاخد ملف وتعطي ملف جديد:

المرحلة	الأمر	الملف الناتج	شنو تدير
1. المعالج المسبق	`gcc -E`	`hello.i`	يلصق الهيدرز ويبدل الماكرو
2. الترجمة	`gcc -S`	`hello.s`	يحول C لـ assembly
3. التجميع	`gcc -c`	`hello.o`	يحول assembly لـ binary (ELF)
4. الربط	`gcc hello.o -o hello`	`hello`	يربط مع مكتبة C ويعطي البرنامج

> **تشبيه بسيط**
> تخيل تبني دار: الـ preprocessing هو تحضير المواد (تلصق كل ما تحتاجو)، والترجمة هي كتابة المخطط، والتجميع هو صنع القطع، والربط هو تركيب القطع مع بعضها باش تولي دار كاملة جاهزة للسكن.

3.2 المرحلة 1: المعالج المسبق (Preprocessing)

المعالج المسبق يقرا السطور اللي تبدا بـ # ويدير التبديلات قبل الترجمة. ما يفهمش C، غير يبدل نصوص.

-
-

> **علاش hello.i طويل؟ وعلاش GREETING اختفى؟**
> hello.c فيه 7 سطور، أما hello.i فيه 845. السبب أن stdio.h فيه آلاف الأسطر من التعريفات، وكلها تتلصق.
> GREETING اختفى لأن المعالج بدلو بالنص في كل مكان. #define ماشي متغير وقت التشغيل، راهو نص يتبدل قبل الترجمة.

3.3 المرحلة 2: الترجمة إلى Assembly

يتحول الكود لتعليمات قريبة من معالج الكمبيوتر، مكتوبة كنص يقراه الإنسان. تشوفو في hello.s:

-
-
-

3.4 المرحلة 3: التجميع (Assembling)

يتحول assembly لملف object (hello.o)، بصيغة ELF، فيه الكود بالأرقام الثنائية. لكن راهو ناقص: puts ما تعرفش فين تتلقى، فتبقى U (undefined).

-
-

3.5 المرحلة 4: الربط (Linking)

الـ linker هو اللي يكمل الناقص. يلقى puts في مكتبة C المشتركة (libc)، ويربطها بالبرنامج، ويعطيك ملف تنفيذي.

-
-

3.6 أسئلة تتكرر في الامتحان

> **hello وماشي hello/.**
> Linux ما يدورش على البرامج في المجلد الحالي تلقائيا، لأسباب أمنية. ./ تقول له: شغل الملف اللي هنا.

o- بدون gcc hello.c
يعطي الملف اسم افتراضي a.out. نكتبو o hello- باش نتحكمو في الاسم.

mpicc
mpicc ماهوش مترجم جديد. هو غلاف (wrapper) حول gcc، يزيد المسارات والمكتبات الخاصة بـ MPI. باش تشوف الأمر الكامل، نستعملو mpicc --showme.

الجزء 4: السكربتات والصلاحيات

4.1 شنو هو السكربت؟

السكربت ملف نصي فيه أوامر الـ terminal، تتنفذ وحدة بوحدة. بدل ما تكتب 4 أوامر كل مرة، تكتبهم في ملف وتشغلو بأمر واحد.

الأمر	الوظيفة
`touch ex01.sh`	ينشئ ملف فارغ
`cat ex01.sh`	يعرض محتوى الملف
`chmod +x ex01.sh`	يعطي صلاحية التنفيذ
`./ex01.sh`	يشغل السكربت

4.2 سطور مهمة في السكربت

-
-
-

4.3 فهم rwxr-xr-x-

الـ 10 حروف تتقسم هكذا: الحرف الأول نوع الملف (- ملف عادي، d مجلد)، ثم 3 مجموعات من 3 حروف: المالك، المجموعة، والباقي.

```
-  rwx  r-x  r-x
   |   |    |    +-- الباقي (others)
   |   |    +------- المجموعة (group)
   |   +------------ المالك (owner)
```

الحرف	المعنى	القيمة الرقمية
r	قراءة (read)	4
w	كتابة (write)	2

الحرف	المعنى	القيمة الرقمية
x	(execute) تنفيذ	1
-	ما عندوش الصلاحية	0

حساب 755

(كل الصلاحيات للمالك) 1+2+4 = 7

(قراءة وتنفيذ للمجموعة والباقي) 1+4 = 5

`chmod +x` يزيد صلاحية التنفيذ، والنتيجة قريبة من 755.

4.4 ملاحظة مهمة: ما تشغلش السكربت من Windows

السكربتات تخدم في مسار Linux فقط (/home/.../). إذا ظهرت لك rwxrwxrwx- فأنت في مجلد Windows، والـ chmod ما يخدمش هنا.

الجزء 5: التمرين 2، argc و argv

لما تكتب أمر في الـ terminal، كل كلمة تتبعث للبرنامج. البرنامج يستقبلها في main:

```
int main(int argc, char *argv[])
```

الاسم	المعنى	مثال
`argc`	عدد الكلمات (بما فيها اسم البرنامج)	args_test 42 → argc = 2/.
`argv[0]`	اسم البرنامج	`"./args_test"`
`argv[1]`	أول وسيطة، نص دائما	`"42"`

القاعدة الذهبية

قبل ما تستعمل argv[i]، لازم تتأكد أن i < argc. وإلا تقرا خارج المصفوفة وتطيح البرنامج (Segmentation fault).

argv[argc] موجود دائما ويساوي NULL، آخر عنصر فارغ.

42 نص وماشي عدد

argv[1] + 1 ما يعطيش 43. يعطي الحرف اللي بعد الأول: "2"، لأن "42" مصفوفة حروف.

لازم تحوّلو لعدد باش تحسبو بيه، وهذا يجي في التمرين 3.

علاش MPI يحتاج argc و argv؟ لأن MPI_Init(&argc, &argv) يقرا منهم معلومات يحتاجها لتشغيل البرامج على عدة معالجات، وهذا تشوفوه في Lab 3.

الجزء 6: التمرين 3، atoi و strtol

الدالة	شنو ترجع	هل تكشف الخطأ؟
`atoi("42")`	`42`	لا
`atoi("abc")`	`0`	لا (خطر)
`atoi("12abc")`	`12`	لا (خطر)
`strtol(...)`	العدد + مؤشر لوين وقف	نعم

> **علاش atoi خطر؟**
> ما تعرفش واش المستخدم كتب 0 بصح، ولا كتب حاجة غلط. الاثنين يرجعو 0.

strtol: ثلاث وسائط

-
-
-

مثال كامل لكشف الخطأ

```
char *end;
long v = strtol(argv[1], &end, 10);
if (*end != '\0') {
    printf("ليس عددا صحيحا\n");
}
```

إذا كان *end ماشي '\0'، معناها بقى شي حروف ما تحولتش، يعني الإدخال غلط.

الجزء 7: التمرين 4، الماكرو

7.1 علاش الأقواس مهمة؟

```
#define SQUARE(x)      x * x
#define SQUARE_OK(x)   ((x) * (x))
```

الاستدعاء	التبديل	النتيجة (N=10)
`SQUARE(N + 1)`	`N + 1 * N + 1`	□ 21

الاستدعاء	التبديل	النتيجة (N=10)
`SQUARE_OK(N + 1)`	`((N + 1) * (N + 1))`	121

> **القاعدة**
> حط الأقواس على كل parameter، وعلى الماكرو كامل. الماكرو يبدل نص، ماشي تعبير رياضي.

7.2 الشروط والتعريف من سطر الأوامر

-
-
-

7.3 الماكروات المدمجة

-

ماشي متغيرات، المترجم يعطيهم وحدو. أسماؤهم تبدا بـ __ لأن هذي الأسماء محجوزة للمترجم والنظام، فما تبداش أسمائك بيهم.

7.4 مزايا كل طريقة

الطريقة	المزية
قيمة بـ -D (وقت الترجمة)	أسرع، والمترجم يحسن الكود حسب القيمة
قيمة من argv (وقت التشغيل)	مرونة أكبر، تغير القيمة بدون إعادة الترجمة

الجزء 8: الأخطاء الشائعة وحلولها

الخطأ	السبب	الحل
`Permission denied`	السكربت ما عندوش صلاحية تنفيذ	`chmod +x ex01.sh`
`/bin/bash^M: bad interpreter`	ملف فيه نهايات Windows (CRLF)	`sed -i 's/\r$//' ex01.sh`
`missing separator`	Makefile فيه مسافات بدل Tab	استعمل Tab في بداية أسطر الأوامر
`file not found (.vbox)`	الملف انتقل أو ما تفكش	تأكد من مكان الملف وأعد الإضافة
`Segmentation fault`	استعمال argv[i] خارج الحدود	تأكد من argc قبل الاستعمال
`cannot find or open lab01_resources.zip`	الملف في مكان آخر	`find ~ -name "lab01_resources.zip"`

> **نصيحة**

أغلب مشاكل المختبر من الملفات (المكان، الاسم، الصلاحيات). قبل أي حاجة، تأكد بـ ls -l و pwd.

الجزء 9: قاموس المصطلحات

المصطلح	المعنى بالدارجة
Preprocessor	المعالج المسبق، يبدل # قبل الترجمة
Compiler	المترجم، يحول C لـ assembly ثم لـ object
Assembly	لغة قريبة من الآلة، كنص مقروء
Object file (.o)	ملف binary ناقص، يستنى الربط
Linker	اللي يربط الملفات والمكتبات مع بعض
libc	مكتبة C الأساسية (فيها printf وputs)
ELF	صيغة الملفات التنفيذية في Linux
argc / argv	عدد الوسائط / مصفوفة الوسائط
Shebang	أول سطر في السكربت (#!/bin/bash)
Macro	تعريف يتبدل بنص قبل الترجمة

الجزء 10: اختبر روحك

جاوب على هذي الأسئلة بكلماتك في دفترك، وبعد شوف الجواب في الجزء الموافق. إذا ما عرفتش الجواب، ارجع للجزء المعني.

- شنو الفرق بين gcc -S و gcc -c؟
- علاش nm hello.o يورينا puts كـ U، وnm hello يورينا puts@GLIBC_2.2.5؟
- علاش SQUARE(N+1) تعطي نتيجة غلط؟
- شنو تدير السطر set -e في السكربت؟
- شنو يعني -rw-r--r-- وشنو الرقم اللي يقابل -rw؟
- علاش لازم نتأكد من argc قبل argv[i]؟
- علاش atoi("abc") خطر؟

أمثلة على الأجوبة (للتأكد بعد ما تجاوب)

gcc -S تولي hello.s (assembly)، وgcc -c تولي hello.o (object). الفرق أن -c ما يربطش.

hello.o فيه puts ناقصة، وبعد الربط يلقاها الـ linker في libc.

لأن الماكرو يبدل نص: N + 1 * N + 1 = 21 بدل 121.

يوقف السكربت أول ما يفشل أمر.

ملف عادي، المالك يقرا ويكتب، والباقي يقراو فقط. 6 = -rw.

لأن i ممكن يكون أكبر من عدد الكلمات، فتقرا خارج المصفوفة.

يرجع 0 بلا ما يقول أنو فيه خطأ، فما تعرفش واش هو 0 حقيقي.

الجزء 11: أمثلة كود للفهم

هذي أمثلة صغيرة باش تفهم الفكرة. هي مش نفس برامج التمارين، ماشي حل للـ TP. التمارين نفسها كتبهم بيدك، باش تفهم وتتقيم.

نصيحة للمراجعة

اقرا المثال، وبعدها سكّر الملف واكتب نسخة جديدة بيدك، وبدّل الأسماء والقيم. هكذا تتعلم.

11.1 الترجمة خطوة بخطوة

مثال برنامج يجمع عددين، نترجموه بالمراحل الأربع:

```
// sum.c
#include <stdio.h>

int main(void) {
    int a = 3, b = 4;
    printf("%d\n", a + b);
    return 0;
}
```

```
gcc -E sum.c -o sum.i      # 1. المسبق المعالج
gcc -S sum.i -o sum.s      # 2. C -> assembly
gcc -c sum.s -o sum.o      # 3. assembly -> object
gcc sum.o -o sum           # 4. الربط
./sum                      # يطبع 7
```

لاحظ أن gcc -Wall -Wextra ينبهك على الأخطاء المحتملة. دير هذا دائما باش تتعود على التحذيرات.

11.2 قراءة الوسائط من سطر الأوامر

مثال برنامج يقرا عددين من المستخدم ويجمعهم. لاحظ الفحص قبل الاستعمال:

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc < 3) {                    // نستعمل ما قبل نتأكد argv
        printf("usage: %s a b\n", argv[0]);
        return 1;
```

```
    }
    int a = atoi(argv[1]);
    int b = atoi(argv[2]);
    printf("%d\n", a + b);
    return 0;
}
```

```
./sum2 5 6      # يطبع 11
./sum2          # يطبع usage (لأن argc = 1)
```

لاحظ: return 1 تعطي كود خروج غير 0، وهذا يعني أن البرنامج فشل. الـ shell يقدر يعرفو بـ echo $?.

11.3 كشف الخطأ بـ strtol

مثال: نتأكد أن الإدخال عدد صحيح كامل قبل ما نستعملو:

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc < 2) return 1;
    char *end;
    long n = strtol(argv[1], &end, 10);
    if (*end != '\0' || end == argv[1]) {
        printf("صحيحا عددا ليس: %s\n", argv[1]);
        return 1;
    }
    printf("العدد = %ld\n", n);
    return 0;
}
```

```
./check 25      # العدد = 25
./check 2x5     # صحيحا عددا ليس: 2x5
./check abc     # صحيحا عددا ليس: abc
```

end == argv]1[تعني أنه ما تحولش ولا حرف واحد، وهذا مهم مع النص الفارغ.

11.4 الماكرو مع الأقواس

```
#include <stdio.h>

#define MAX(a, b) ((a) > (b) ? (a) : (b))

int main(void) {
#ifdef DEBUG
    printf("debug mode, file %s, line %d\n", __FILE__, __LINE__);
#endif
    printf("%d\n", MAX(3, 8));        // 8
    printf("%d\n", MAX(2 + 2, 3));    // 4
    return 0;
```

```
}
```

```
gcc -Wall -Wextra demo.c -o demo
gcc -Wall -Wextra -DDEBUG demo.c -o demo    # سطر يفعل debug
```

بدون الأقواس حول a و b، MAX)2 + 2, 3(تولي خدمة غلط. دائما حط الأقواس على كل parameter وعلى الماكرو كامل.

11.5 هيكل سكربت (template)

هذا هيكل عام تقدر تعاود تستعملو لأي سكربت. بدل الأوامر بأوامرك:

```
#!/bin/bash
# للسكربت قصير وصف
set -e                              # خطأ أول عند يوقف

echo "=== الخطوة 1"
gcc -Wall -Wextra demo.c -o demo

echo "=== البرنامج تشغيل"
./demo
```

```
chmod +x demo.sh     # التنفيذ صلاحية
./demo.sh            # التشغيل
```

echo تطبع رسالة توضح لك أي جزء من السكربت شغال، وهذا مفيد لما يكون السكربت طويل.